

- 5% in class exercises
- 10% project
- 40% MIDTERM
- 40% Final
- 5% Labs

Scheme

- Functional

- Primitives:

Integers, floats
 strings: "denia"
 booleans #t, #f
 procedures: AND/OR etc
 string=? string-length

- Combinations: (Expr w. procedure value arg1 arg2)

ex. (+ 2 3) $\Rightarrow$ 2+3
 (square x) $\Rightarrow$ x^2

- Abstraction (define foo 42)

- Interpreter: ^{special}

if primitive, ret value
 elif name (var), ret associated value [+, - ... are names]
 elif special form (ex. define) do smth special
 elif combination, 1. evaluate all subexpressions
 2. apply the operator to args

↳ if primitive do it

elif lambda evaluate body
 with given arguments

~~define~~ Lambda
~~foo (lambda (x) (x x x))~~

Lists

Pairs

Trees

(list ---)

$\equiv '(- - -)$

$(- \cdot -)$

car $\rightarrow$ 1 element

cdr $\rightarrow$ everything except 1 elem
(rest)

lists are
pairs.

(1 2 3)

cons $\rightarrow$ add arg[1]
to list (arg[2])

$\equiv$

(1.(2.(3.())))

null? $\rightarrow$ is list
empty

null $\equiv '()$

(c---r list)

add
add
etc

as much as you like
for indexing.

Definition Style

① Top down

$A \in \text{LoI}$ iff 1. $A = ()$ OR

2. A is a pair whose car
is an int & cdr is LoI

② Bottom up

LoI is the smallest 1. $() \in \text{LoI}$

set that satisfies 2. $(n.e) \in \text{LoI}$ where

③ Rules of Inference

Notation: $\frac{\text{Antecedents}}{\text{Consequents}} \Rightarrow$

$\frac{}{() \in \text{LoI}} \quad \frac{n \in \text{int} \quad e \in \text{LoI}}{(n.e) \in \text{LoI}}$

Axiom

④ Grammars

$LoI ::= ()$

$LoI ::= (Int, LoI)$

Notation:

NonTerminal
symbol
or start symbol

terminal
symbol
or non
terminal
symbol

Thm: A bintree an odd # nodes

$BinTree ::= Int$

$:= (Symbol BinTree BinTree)$

→ Induction

Proof: Base case: $Int \Rightarrow odd \checkmark$

Assume: $(Symbol BinTree BinTree)$
is odd.

then, a smaller BinTree is also
odd, an Int is odd too. Then,

$(Symbol BinTree BinTree)$ is
 $\downarrow \quad \quad \downarrow \quad \quad \downarrow$
 $1 \quad \quad odd \quad \quad odd$ odd.
~~##~~

Data Abstraction

Notation: $\Gamma v \Gamma$ representation of v (data)

~ Interface: $(zero) = \Gamma 0 \Gamma$

$(is-zero? \Gamma n \Gamma) = \begin{cases} 1, & n=0 \\ 0, & n \neq 0 \end{cases}$

$n \geq 0 \quad \left\{ \begin{array}{l} (successor \Gamma n \Gamma) = \Gamma n+1 \Gamma \\ (predator \Gamma n+1 \Gamma) = \Gamma n \Gamma \end{array} \right. \Rightarrow \text{natural numbers}$

Representing v :

□ UNARY

$\Gamma 0 \Gamma = ()$

$\Gamma n \Gamma = (\#t \#t \dots \#t)$
 n

(def (empty-env) (empty-env))

(def (extend-env var val env) (list extend-env var val env))

(def (apply-env env var) (<ICE>))

□ Procedural representation

(define (apply-env env var) (env var))

(define (empty-env) (lambda (var) (error)))

Int as lambda - Church numerals

A tool for defining
Data types

□ Constructors

□ Predicates

~~terms~~ is-smth?

□ Extractors

car, cdr etc

Tool Interface

(define-datatype ^{symbol} lc-exp ^{symbol} lc-exp?)

Constructors

① (var-exp (var identifier?))

② (lambda-exp (^{arguments} bound-var identifier?)
(body lc-exp?))

③ (app-exp (rator lc-exp?)
(rand lc-exp?)))

argument types

Declares
3 constructors
var-exp
lambda-exp
app-exp

1 predicate
lc-exp?

for $lc\text{-}exp ::= \text{Identifier} \text{ ①}$
 $(\text{lambda } (\text{Identifier}) \text{ } lc\text{-}exp) \text{ ②}$
 $(lc\text{-}exp \text{ } lc\text{-}exp) \text{ ③}$

Representation

$(\text{define } a \text{ (var-exp 'bar)}) \rightarrow lc\text{-}exp$
 $(\text{define } b \text{ (lambda-exp 'baz' (app-exp f 'x))})$
 $(\text{define } c \text{ (app-exp b a)})$ valid lc-exp examples
 $(lc\text{-}exp? a) \Rightarrow \#t$

Tool #2

$(\text{define occurs-free (lambda (var exp)$

$(\text{cases } lc\text{-}exp \text{ exp}$

$(\text{var-exp } (var1) \text{ (eq? var1 var)})$

$(\text{lambda-exp } (var1 \text{ body}))$ subtype args used during construction

$(\text{and (not (eq? var1 var))$

$(\text{occurs-free? var body})) \dots$

$(\text{app-exp } \dots$

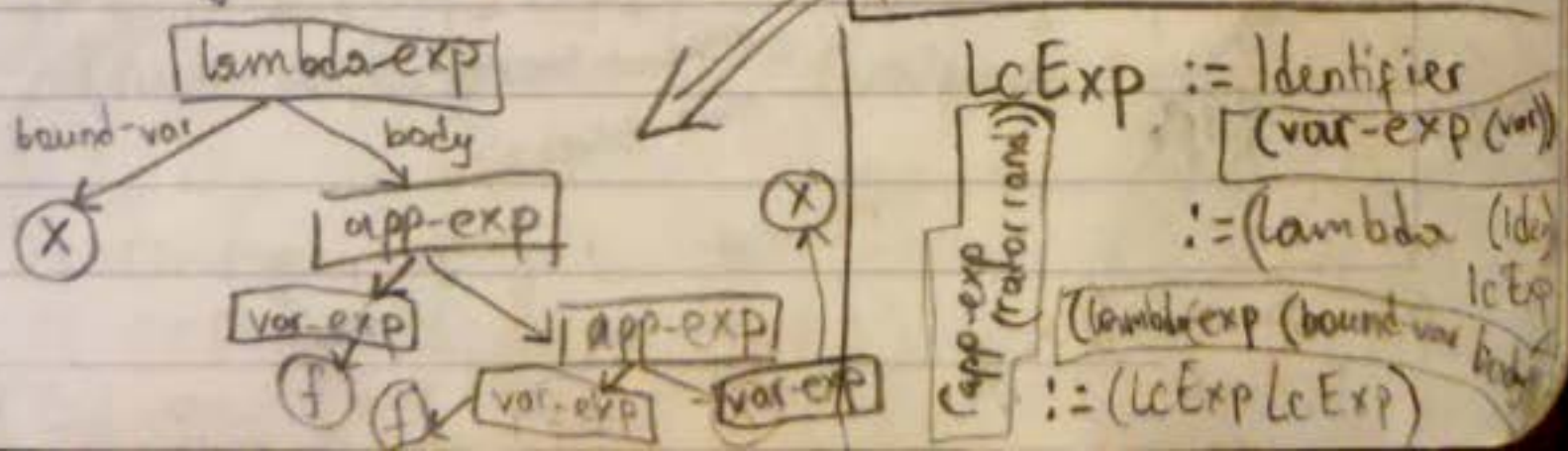
$)$

$)$

Subtypes (indicated by a bracket on the left side of the cases block)

New Languages

Abstract syntax tree



(define parse ;; Scheme $\rightarrow$ LcExp

(λ (datum)

(cond ((symbol? datum) (var-exp datum))

((pair? datum)

(if (eq? (car datum) 'lambda)

(λ -exp " in ice)

(define unparse ;; LcExp $\rightarrow$ Scheme

(λ (exp)

(cond ((eq? (car exp) 'var-exp) (cadr exp))

((eq? (car exp) 'lambda-exp) (cons '()

(cons '(lambda) (cons (cadr exp)

(unparse (caddr exp))))

((eq? (car exp) 'app exp) (cons

(unparse (cadr exp)) (unparse (caddr exp))

LET (a new language)

$\lambda \rightarrow (\arg 1, \arg 2)$

if $\lambda \rightarrow (\arg)$ then x else y

let identifier = proc1(arg1, arg2) in proc2(identifier, arg3)
↳ returns value of proc2. Either outputs int or bool

Exp := Number | const-exp (num)

:= $-(Exp, Exp)$ | diff-exp (exp1, exp2)

:= zero?(Exp) | zero-exp (exp)

:= if Exp then Exp else Exp | if-exp (cond, exp1, exp2)

:= Identifier | var-exp (var)

:= let Identifier = Exp in Exp | let-exp (var, exp1, exp2)

(define-datatype exp exp?)

(const-exp (num num?))

(diff-exp* (exp1 exp?) (exp2 exp?))

...

define-datatype expval expval?

(num-val (value num?))

(bool-val (value bool?))

string

exp

expval

(define (values exp env) (cases expression exp

(const-exp (num) (num-val num))

(var-exp (var) (apply-env env var))

(diff-exp (exp1 exp2) (values exps env))
(num-val (- (exval $\rightarrow$ num
(expval $\rightarrow$ num
(values exp2 env))))

(zero-exp (exp1) ... ice)

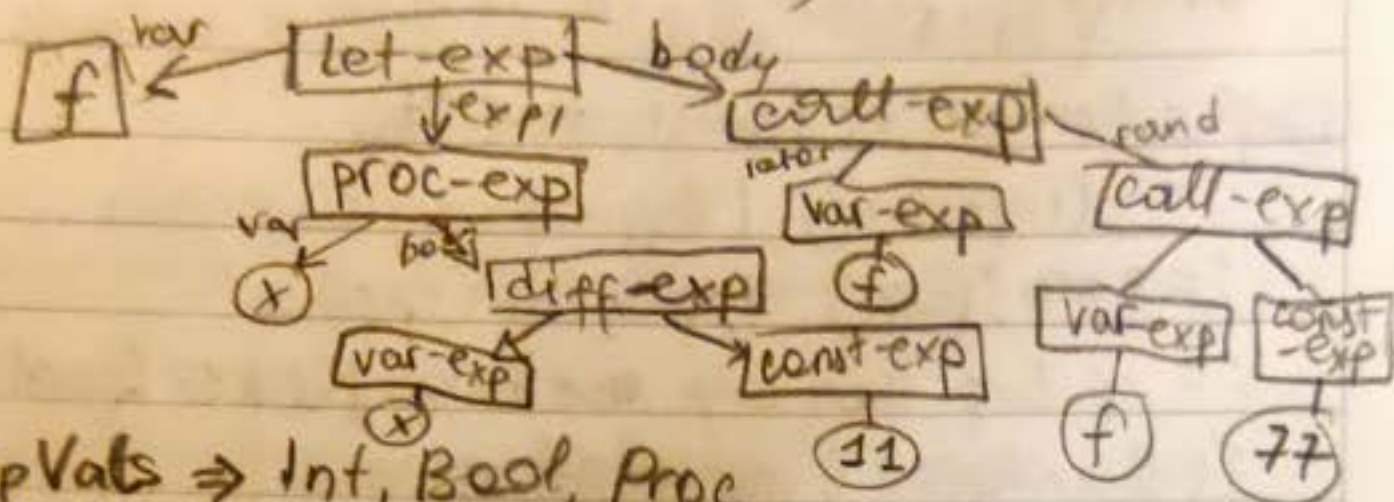
(if-exp (exp1 exp2 exp3) ... ice)

(let-exp (var exp1 body) (value of ... (env ...))

PROC (an extension to LET)

Let $f = \text{proc}(x) \rightarrow (x, 11)$ in $(f(f 77)) \rightarrow 55$

$(\text{proc}(f) (f(f 77)) \text{proc}(x) \rightarrow (x, 11)) \rightarrow 55$



1) New ExpVals $\Rightarrow$ Int, Bool, Proc

2) Grammar addition

Expression $:= \text{proc}(\text{Identifier}) \text{Exp} \mid \text{proc-exp}(\text{var}, \text{body})$
 $:= (\text{exp exp}) \mid \text{call-exp}(\text{rator}, \text{rand})$

3) Specification and Implementation

(value-of (proc-exp var body) ρ)
 $= (\text{proc-val} (\text{procedure var body } \rho)) \leftarrow$
 (value-of (call-exp rator rand) ρ)
 $= (\text{let } ((\text{proc} (\text{expval} \rightarrow \text{proc} (\text{value-of rator } \rho)))$
 $(\text{arg } (\text{value-of rand } \rho)))$
 $(\text{apply-procedure proc arg}))$

4) Representation

(define procedure (λ (var body) ρ)

OR (value-of body [var=val] ρ))

(define-datatype proc proc?
 procedure (var-identifier?)
 (body expression?)
 (env environment?))

environment where the procedure was defined, not the current environment

~ Currying

Let $f = \text{proc}(x) \text{proc}(y) \dots$
IN $((f\ 3)\ 4) \dots$

~ Recursion

let $f = \text{proc}(x) \text{ if zero?}(x) \ x \text{ else } (f\ -(x, 1))$
in $(f\ 3)$

does not work: f is not defined in the environment of the thing
 f is only defined (scope!) in the body of the let exp.

"Lambda combinator"

let $f = \text{proc}(\text{maker}) \text{proc}(x) \text{ if zero?}(x) \ x$
in $((f\ f)\ 3)$ else $(f\ -(x, 1))$

works, but too verbose

Let REC (an extension to

letrec double $(x) =$

if zero?(x)

then 0

else $-((\text{double } -(x, 1)), -2)$

in double 6)

LETREC

letrec double(x) = if zero?(x) then 0 else +((double -(x, 1)), 2)
in (double 6)

- Grammar

Exp := letrec Identifier (Identifier) = Expr in Expr

[letrec-exp (pname bvar pbody letrec-body)]

- Specification

(value-of (letrec-exp pname bvar pbody letrec-body) ρ)

= (value-of letrec-body, (extend-env-rec pname bvar pbody ρ))

Assume (extend-env-rec) returns ρ_1

(apply-env ρ var) $\rightarrow$ if var = pname $\rightarrow$ (proc-val (procedure
else (apply-env ρ_0 var) bvar pbody ρ_1))

- Implementation

(define-datatype environment env?

(empty-env)

(extend-env (var identifier?)

(val exp-val?)

(env env?))

(extend-env-rec (pname identifier?)

(bvar identifier?)

(pbody exp?)

(env env?))

(define apply-env (λ (env var)

(cases environment env

(empty-env () error)

(extend-env (var1 val1 env0)

(if (eq? var1 val1) val1

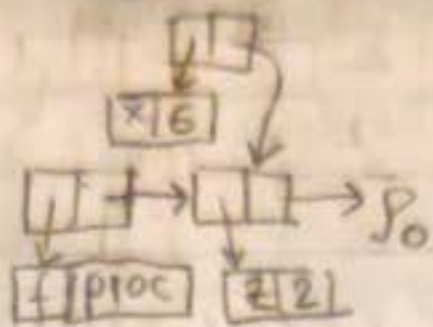
(apply-env env0 var1))

(extend-env-rec (pname bvar pbody env0)

(if (eq? var pname) (proc-val (procedure pname bvar pbody env0))

((if (var env var)

Environment diagram



Binding & Scoping

(lambda (x) (+ x 3))

~~~~~  
declaration      ~~~~~  
reference

$(\text{let } ((x (+ y 7))) (+ x 3))$   
 $\downarrow$  declaration  $\rightarrow$  reference

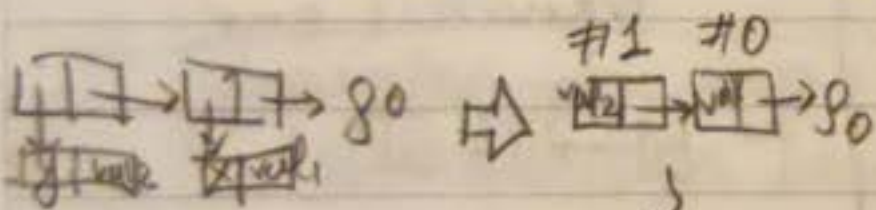
$(\text{let } ((x\ 3)\ (y\ 4))\ (+\ (\text{let } ((x\ (+\ y\ 5))))\ (*\ x\ y)\ x))$   
 scope of  
 $x = y + 5$

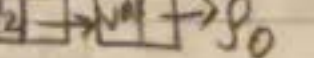
Binding:  $\lambda(x) \dots$   
 ↓  
 does not have a binding yet  
 (only when called)

## Lexical addressing

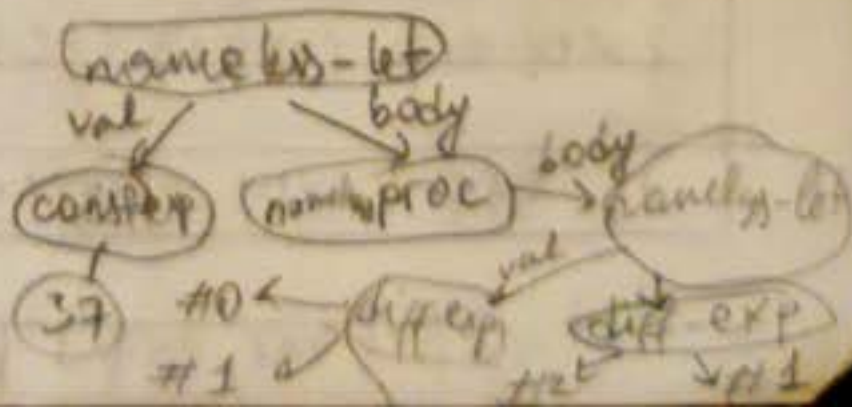
$$(\text{lambda } (x_i) \quad ((\text{lambda } (a_i) \quad [x \ a]) \ x))$$

$\leadsto (\lambda (x) (\lambda (y) (x \# y))) \# 0$




 are the lexical addresses  
 no need to store  
 var names! can find with the  
 lexical index

```
Ex    let 37
        in proc
          let -(#0, #1)
          in -(#2, #1)
```



□ Translator

static env

```

(define (translation-of exp env)
  (cases expression exp
    (const-exp (num) (const-exp num))
    (diff-exp (exp1 exp2) (diff-exp (translation-of exp1 env)
                                       (translation-of exp2 env)))
    (var-exp (var) (namelet-var-exp (
    apply-env env var))))
  )

```

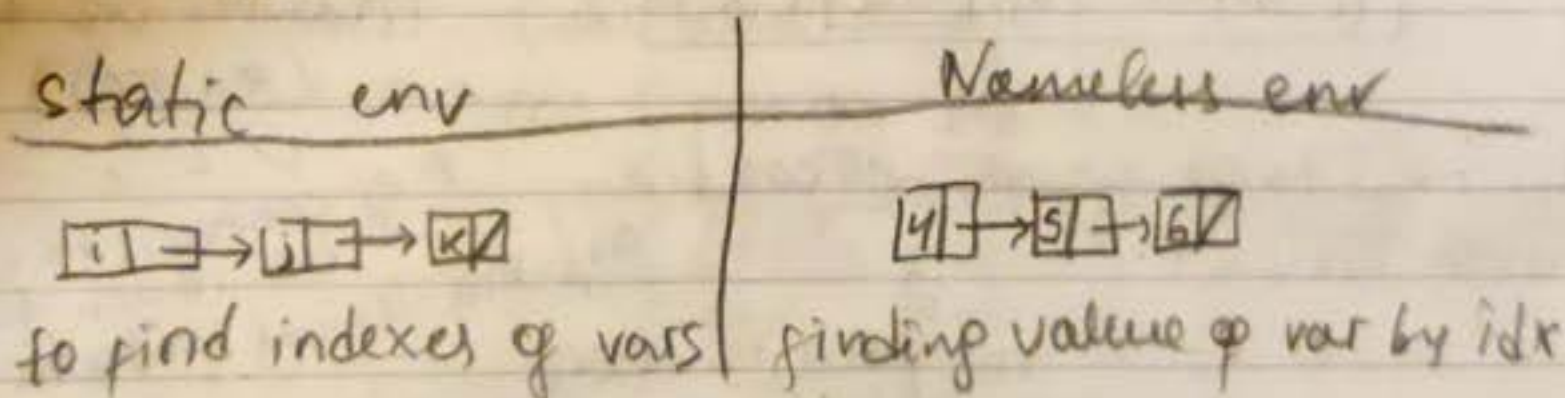
□ Static env (list of var names)

```

(define empty-env (lambda () '()))
(define extend-env (lambda (var env) (cons var env)))
(define apply-env (lambda (env var) (cond ((null? env)
                                           (error "no binding for " var))
                                           ((eq? var (car env)) 0)
                                           (else (+ 1 (apply-env (cdr env) var)))))

```

→ inserting var into static env



□ Proc translation

```

zero? exp (exp1) (zero-exp (trans exp1) env)
if-exp (exp1, exp2, exp3) (if-exp (trans exp1 env) (trans exp2 env) (trans exp3 env) ...)
let-exp (var, exp1 body) (namelet-let-exp (trans exp1 env) (trans body (extend-var env var)))

```

$\text{proc-exp (var body)}$   $\left( \begin{smallmatrix} \text{nameless-proc-exp.} \\ \text{(trans body (extend-env var env))} \end{smallmatrix} \right)$   
 $\text{call-exp (rator rand)}$   $\left( \begin{smallmatrix} \text{call-exp} \\ \text{(trans rator} \\ \text{env)} \end{smallmatrix} \right) \left( \text{trans rand} \\ \text{env} \right)$

☐ Nameless Env (list of values) 
 $\begin{matrix} \#0 & \#1 & \#2 \\ \boxed{\text{val}} \rightarrow \boxed{\text{val}} \rightarrow \boxed{\text{val}} \end{matrix}$

(define (empty-nameless-env) => '())

(define (extend-nameless-env val env) => cons

(define (apply-nameless-env idx env) => cdr until idx  
is 0, car.

☐ Interpreter

(define (nameless-value-of exp nenv)

(cases expression exp

(const-exp (num) (numval num)

(nameless-var-exp (idx) (apply-nenv idx nenv))

(nameless-let-exp (exp1 body)

(nameless-val-of body (extend-nenv  
(apply-nenv exp1 nenv)  
nenv)))

(nameless-proc-exp (body)

(nameless-proc body nenv)

nameless-proc (body env)

(lambda (rand) (nameless-value-of body (extend-env  
rand env)))

Recall:

$E_{Ref}$ 

Memory:  $\sigma$   store

Eref: Exprval := Int

Boat

РГОС

Ref(ExpVal)  $\Rightarrow$  pointer

newref (val)  $\Rightarrow$  alloc, put val there,  
return address

deref(ref)  $\Rightarrow$  get value from address

setref(ref, val)  $\Rightarrow$  change the value at address

let  $x = \text{newref}(0)$

in tetrec even  $(-)=$

if zero? (deref(x))

then 1

else begin

$$\text{setref}(x, -(\text{derref}(x), 1))$$

(odd —)

end

$$\text{odd}(-) =$$

if zero? (defeq (x))

then 0

else begin

$$\text{Setref}(x, -(\text{derog}(x), 1))$$

(even —)

end

```
in begin setref(x, 13).  
    odd(-) end
```

odd (-) end

let  $g =$

(b) counter newref(0)

in p f e c (12)

```
begin setref(counter, -(defref(counter) - 1))
  deref(counter)
```

deret (counter)

end

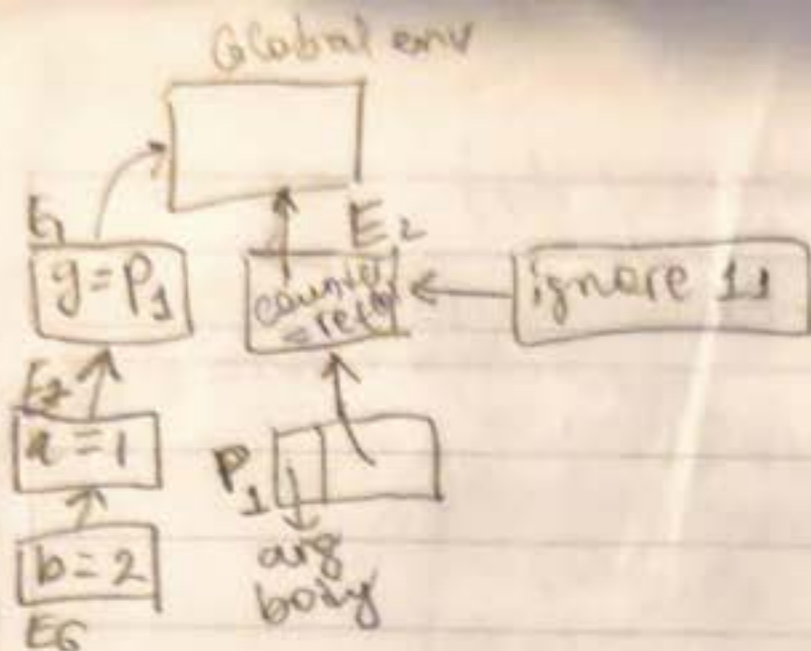
in let  $q = (q, 1)$

in)  $b + b = (g \parallel)$

$$\ln \boxed{-(a, b)}$$

Global env

arg. ignore  
body. begin...end  
entire never  
change



## □ Specification

$f: env \rightarrow [v=0]f$  modified env

$\sigma: store \rightarrow [x=0]\sigma$  modified store

(value-of exp  $f, \sigma_0$ ) = (val1  $\sigma_1$ )

(value-of (diff-exp exp1 exp2)

= (value-of exp1  $f, \sigma_0$ )  $\Rightarrow$  (val1  $\sigma_1$ )

(value-of exp2  $f, \sigma_1$ )  $\Rightarrow$  (val2  $\sigma_2$ )

$\Rightarrow$  (val1 - val2,  $\sigma_2$ )

(value-of (newf exp)  $f, \sigma_0$ )  $\Rightarrow$  ((ref-val  $l$ ),  $l \notin \text{dom}(\sigma_1)$ )  
 $\downarrow$  evaluate  $\Rightarrow$  (val1,  $\sigma_1$ )  
 $\downarrow$  [l = val]  $\sigma_1$ )

(value-of (desf exp)  $f, \sigma_0$ )  $\Rightarrow$  ( $\sigma_1(l)$ ,  $\sigma_1$ )  
 $\downarrow$  evaluate  $\Rightarrow$  (l,  $\sigma_1$ )  
 $\downarrow$  apply store

## □ Implementation of store

(empty-store)  $\Rightarrow$  ()

(reference? x)  $\Rightarrow$  (integer? x)

(set! x 5)

(set-car! p 5)

(set-cdr! p 5)

!  $\Rightarrow$  in place

```
(define the-store '())  
(define (deref ref) (list-ref the-store ref))  
(define (newref val) (append! the-store (list val)))  
and return size-1
```

## Implementation

(define (value-of exp env)  
 (cases expression exp  
 (const-exp (num) ...  
 ...  
 (newref-exp (exp1) (ref-val (newref (value-of exp1 env))))  
 (deref-exp (exp1) (deref (expval → ref (value-of exp1 env))))  
 (setref-exp (exp1 exp2)  
 ... eval exp1, eval exp2, call setref helper,  
 return old value.

IREF

implicit references

## DenVal vs ExpVal

Ref(ExpVal)      Int, Bool, proc

- Env/store

ENV

| var | val |
|-----|-----|
| x   | ref |

Store  
ref → val

☐ Grammar

Expr := set Identifier = Expr

```
assign-exp(var exp1)
```

Example:  $\text{let } g = \text{let count} = 0 \text{ in } \text{proc } (-) \text{ in } \text{let } a = (g \ 1) \text{ in } \text{let } b = (g \ 1) \text{ in } \text{let count} = -(count, -1); \text{ in } -(a, b)$

## □ Specification:

| - var-exp                                                                                         | - assign                                                                                                                                                                                        | - letrec | - apply-proc                                                                                                                                                                  |
|---------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $(\text{value of } \text{var-exp})$<br>$(\text{var}, \sigma)$<br>$= (\sigma(\text{var}), \sigma)$ | $(\text{value of } \text{assign-exp})$<br>$(\text{var}, \text{exp}, \sigma)$<br>$= (\text{value of exp}, \sigma)$<br>$= (val_1, \sigma_1)$<br>$\Rightarrow (42, [\text{var} = val_1] \sigma_1)$ |          | $(\text{value of } \text{apply-proc})$<br>$(\text{proc var}, \text{body}, \sigma)$<br>$= (\text{value of body}, [\text{var} = \text{val}], [\text{val} = \text{val}] \sigma)$ |

## □ Implementation

(define (value-of exp env)  
 (cases expression exp

(var-exp (var) (deref (apply-env env var)))

(assign-exp (var exp1)

(setref! (apply-env env var) (value-of exp1 env))  
 (numval 42))

(let-exp (var exp1 body)

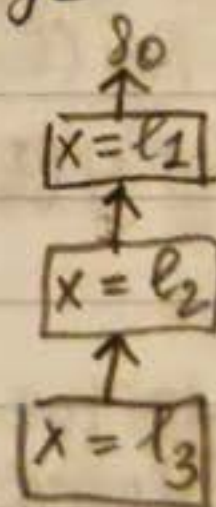
(value-of body (extend-env var (newref env))  
 (value-of exp1 env)))

★ Note: only assign-exp can change value at a location

Example: let x=5 in  
 let x=6 in  
 let x=7 in  
 ...

store

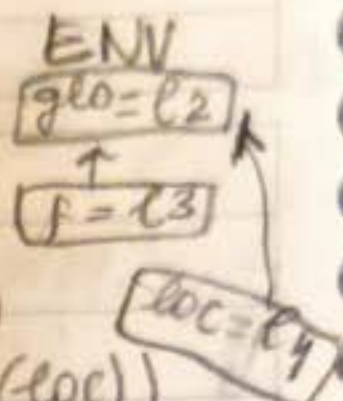
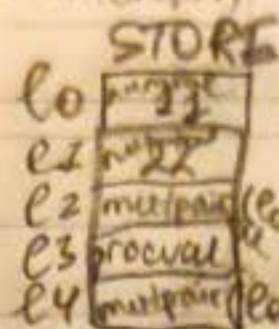
|    |   |
|----|---|
| l1 | 5 |
| l2 | 6 |
| l3 | 7 |



# Mutable Pairs [LREF extension]

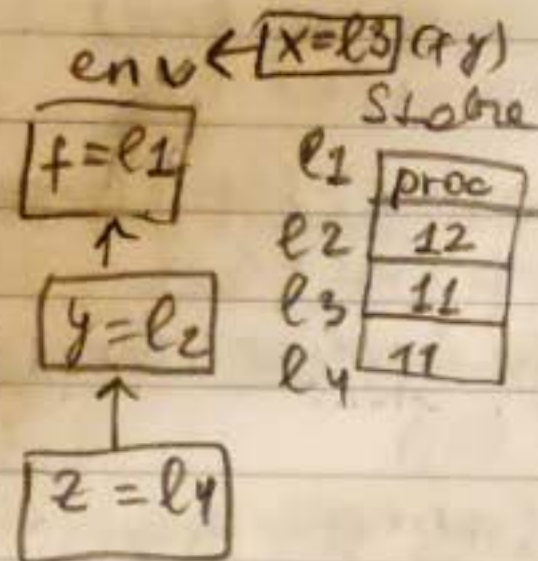
Ex:

```
let glo = pair(11, 22)
in let f = proc(loc)
  let d1 = setright(loc, left(loc))
  in let dz = setleft(glo, 99)
    in - (left(loc), right(loc))
  in (f glo) → 88
```



pair  
left  
right  
setleft  
setright

```
let f = proc(x)
  set(x, -(x, 1))
in let y = 12
  in let z = f(y)
    in y → 12
```



Types

Int  
Bool  
Proc  
Exprval  
Mutpair  
Ref

## Interpreter

Exprval: Int, Bool, Proc, Mutpair

Denvval: Ref(Exprval)

Mutpair: Ref(Exprval) × Ref(Exprval)

## Helpers:

```
(define datatype Mutpair mutpair?
  (pair (left-loc reference?)
    constructor (right-loc reference?)))
```

```
(define makepair (λ (val1 val2)
  pair (newref val1) (newref val2))
```

```
(define left (λ (p) (cases mutpair p
  (pair (left-loc, right-loc) (deref left-loc))))
```

```

(define (setleft p, val) (cases mutpair p
  (pair (leftloc rightloc) (setref! leftloc val))))

(define (valueof exp env) (cases expression exp
  ...
  (pair exp1 exp2) (mutpair-val
    (makepair (valueof exp1 env) (valueof exp2 env))
  (left exp) (left (expval → mutpair (valueof exp1
    ...
    env))))
  (setleft exp1 exp2)
  (let ((val1 (valueof exp1 env)) (val2 (valueof exp2 env))
    (setleft (expval → mutpair val1) val2)
    (mutpair-val val2))))

```

## Parameter Passing

### DEFINITIONS

formal parameter: ~~argument~~ param name in proc

actual parameter: <sup>expression</sup> (argument) passed into proc

denoted value: when proc is called, formal param is bound to a denoted value (a new ref in iref)

Proc: natural parameter passing (denoted value  $\equiv$  expressed value of actual param)

```

(valueof (call exp rator rand) f)
= (let ((proc (expval → proc (valueof rator f))
  (arg (valueof rand f))
  (apply procedure proc arg)

```

```

apply procedure (proc body var, f) val)
= (valueof body [var=val] f)

```

IREF: call by value (denoted value  $\equiv$  reference to expression value)  
 (value of (call-exp ... )  $\rho$ )

$\Rightarrow$  (value of (var-exp var)  $\rho$   $\sigma$ ) = ( $\sigma(\rho(\text{var}))$ ,  $\sigma$ )

(apply-procedure (proc val body  $\rho$ ) val  $\sigma$ )  
 = (value of body [var =  $l$ ]  $\rho$  [ $l = \text{val}$ ]  $\sigma$ )

CBR: call by reference (denoted value  $\equiv$  the reference to the actual param if it is a var else a new location.)  
 e.g. (p a)  $\Rightarrow$  pass ref of a (existing loc)  
 (p -(a, 1))  $\Rightarrow$  same as CBR (old loc)

(value of (call-exp rator rand)  $\rho$ )

= (let ((proc (...)))  
 (ref (value of-operand rand  $\rho$   $\sigma$ )))  
 (apply-procedure proc ref))

$\rightarrow$  (apply-procedure (proc val body  $\rho$ ) val  $\sigma$ )  
 = (value of body [var = ref]  $\rho$ ,  $\sigma$ )

((value of-operand exp  $\rho$   $\sigma$ ) (cases exp exp  
 (var-exp var) ( $\rho(\text{var})$ ,  $\sigma$ )  
 (otherwise) (newref (value of exp  $\rho$   $\sigma$ ), [ $l = \text{val}$ ]  $\sigma$ ))  
 $\downarrow$   
 (val,  $\sigma_1$ ))

# Lazy evaluation

## □ call-by-name

Do not evaluate expression now, do it when needed  
(do not store it anywhere though)

## □ call-by-need

Evaluate when needed and store.

ex:  $\text{proc}(x) \rightarrow (x, x)$   $\rightarrow$  CBName slower  
CBNeed faster.

## □ Lazy streams in lazy evaluation

Let  $\text{cons} = \text{proc}(a) \text{proc}(b) \text{proc}(c)$  if zero?(c) then a  
else b

let  $\text{car} = \text{proc}(p) (p 0)$  in

let  $\text{cdr} = \text{proc}(p) (p 42)$  in

let  $\text{ones} = 1$

in Begin

set  $\text{ones} = ((\text{cons } 1) \text{ ones})$

(car ones)  $\Rightarrow 1$

(car (cdr ones))  $\Rightarrow 1$

end

OR

IN

let  $\text{ints} = 1$

in Begin

set  $\text{ints} = ((\text{cons } 1) (\text{add ones} \text{ ints}))$

(car)

(cdr)  $\Rightarrow 1, 2, 3, 4, \dots$

end

why? because: only  
env is packaged  
Store is changed  
 $\Rightarrow$  ones points to  
the list.

1, 1, 1, ...

$\nwarrow$  infinite  
addition

Letrec  $\text{add}(A) = \text{proc}(B)$

((cons - (car A) - (0, (car B))),

(add (cdr A) (cdr B)))

## Sieve of Eratosthenes

let filter(p) = proc (A)

if (p(car A))

then ((cons(car A)) (filter(cdr A) p))

else ((filter p) (cdr A))

letrec sieve (A) ((cons(car A))

(sieve (filter (NotDiv (car A)) (cdr A))))

↓  
returns proc